COMPARATIVE ANALYSIS OF MACHINE LEARNING AND DEEP LEARNING MODELS FOR NLP

Objective: Implement and compare five models for NLP sentiment classification: Naive Bayes, Support Vector Machine (SVM), Artificial Neural Network (ANN), Recurrent Neural Network (RNN), and Long Short-Term Memory (LSTM). Evaluate them using common classification metrics and visualizations.

1. Experiment Overview

Model	Category	Input representation	Main principle
Naive Bayes	Machine Learning	TF-IDF	Probability-based classification
SVM	Machine Learning	TF-IDF	Finds a separating decision boundary
ANN	Deep Learning	TF-IDF	Learns patterns through dense neural layers
RNN	Deep Learning	Word sequences + embedding	Processes words sequentially using hidden states
LSTM	Deep Learning	Word sequences + embedding	Uses gates and memory to learn sequence dependencies

2. Execution Workflow

Stage	Description
Input dataset	Load labelled customer reviews from sentiment_dataset.csv.
Preprocessing	Clean missing entries, normalize labels and create a stratified train-test split.
TF-IDF	Convert reviews into numerical features for Naive Bayes, SVM and ANN.
Word sequences	Tokenize reviews and create embeddings for RNN and LSTM.
Training	Train all five models using the same training partition.
Evaluation	Calculate accuracy, precision, recall and F1-score on the same held-out test set.
Visualization	Plot model comparison, confusion matrices and deep-learning training loss.

3. Dataset Preparation

Use the previously created sentiment_dataset.csv. It must contain a Review column and a Sentiment column with values Positive or Negative. The practice file has 100 synthetic reviews: 50 positive and 50 negative.

Example records

Review	Sentiment
The product is excellent	Positive
Amazing quality and fast delivery	Positive
Very poor quality	Negative
The service was terrible	Negative

Dataset limitation: This small synthetic dataset is suitable for demonstrating the code and workflow, but it is too small and repetitive to establish reliable real-world performance or a definitive ranking of the five models.

4. Software Requirements

Use Python with pandas, NumPy, scikit-learn, TensorFlow, Matplotlib and Seaborn. CPU execution is sufficient for this laboratory demonstration.

```
python -m pip install --upgrade pip
pip install pandas numpy scikit-learn tensorflow matplotlib seaborn
```

5. Why Two Feature-Preparation Methods Are Used

Models	Representation	Explanation
Naive Bayes, SVM, ANN	TF-IDF	Represents each review using word and word-pair importance scores. The A
RNN, LSTM	Integer sequences + Embedding	Preserves word order. An embedding layer maps token IDs to trainable dens

6. Understanding the Five Models

Model	What it does
Naive Bayes	Estimates class probabilities from features using Bayes' theorem and a conditional-independence assumption.
SVM	Learns a decision boundary that separates positive and negative review feature vectors.
ANN	Uses dense layers and nonlinear activation functions to learn combinations of TF-IDF features.
RNN	Reads tokens sequentially and updates a hidden state as each token is processed.
LSTM	Uses input, forget and output gates with a cell state to help retain useful sequence information.

7. Making the Comparison Fair

All five models use the same stratified training and test split. TF-IDF is fitted only on the training reviews, then applied to the test reviews. The deep-learning text vectorizers are also adapted only on training text. This avoids allowing held-out test text to influence the learned vocabulary or TF-IDF weights.

The comparison is between practical model pipelines, not only between classifier algorithms: TF-IDF is used for the first three models, while sequence embeddings are used for RNN and LSTM. Training time and model complexity should also be considered when interpreting results.

8. Complete Python Implementation - Part 1

Create compare_ml_dl_nlp.py in the same folder as the CSV file. Add the following code first.

```
# COMPARATIVE ANALYSIS OF ML AND DL MODELS FOR NLP

import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report
)

from tensorflow.keras.layers import (
    Input, Dense, Dropout, Embedding, SimpleRNN, LSTM,
    TextVectorization
)
from tensorflow.keras.callbacks import EarlyStopping

# Reproducibility
np.random.seed(42)
tf.random.set_seed(42)

# STEP 1: Load dataset
data = pd.read_csv("sentiment_dataset.csv")
data = data.dropna(subset=["Review", "Sentiment"])

X = data["Review"].astype(str).to_numpy()
y = (
    data["Sentiment"].str.strip().str.lower()
    .map({"negative": 0, "positive": 1})
)

if y.isna().any():
    raise ValueError("Invalid sentiment labels found.")
y = y.to_numpy(dtype="int32")

print("Total reviews:", len(X))

# STEP 2: Common stratified train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

results = []
confusion_matrices = {}
training_histories = {}

# Common evaluation function
def evaluate_model(name, actual, predicted):
    accuracy = accuracy_score(actual, predicted)
    precision = precision_score(actual, predicted, zero_division=0)
    recall = recall_score(actual, predicted, zero_division=0)
    f1 = f1_score(actual, predicted, zero_division=0)

    results.append({
        "Model": name,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-Score": f1
    })

    confusion_matrices[name] = confusion_matrix(
        actual, predicted, labels=[0, 1]
    )

    print("\\n", name, "RESULTS")
    print("Accuracy:", round(accuracy, 4))
    print("Precision:", round(precision, 4))
    print("Recall:", round(recall, 4))
    print("F1-score:", round(f1, 4))

    print(classification_report(
        actual, predicted,
        labels=[0, 1],
        target_names=["Negative", "Positive"],
        zero_division=0
    ))
```

8. Complete Python Implementation - Part 2

Continue the same Python file with the following code.

```
# STEP 3: TF-IDF feature extraction for NB, SVM and ANN
# Keep negation terms such as "not" and "no" in the vocabulary.
tfidf = TfidfVectorizer(
    max_features=2000,
    ngram_range=(1, 2),
    lowercase=True
)

X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

# STEP 4: NAIVE BAYES
print("\\nTRAINING NAIVE BAYES")
nb_model = MultinomialNB()
nb_model.fit(X_train_tfidf, y_train)
nb_pred = nb_model.predict(X_test_tfidf)
evaluate_model("Naive Bayes", y_test, nb_pred)

# STEP 5: SUPPORT VECTOR MACHINE
print("\\nTRAINING SVM")
svm_model = LinearSVC(random_state=42)
svm_model.fit(X_train_tfidf, y_train)
svm_pred = svm_model.predict(X_test_tfidf)
evaluate_model("SVM", y_test, svm_pred)

# STEP 6: ARTIFICIAL NEURAL NETWORK (ANN)
print("\\nTRAINING ANN")
X_train_dense = X_train_tfidf.toarray().astype("float32")
X_test_dense = X_test_tfidf.toarray().astype("float32")

ann_model = tf.keras.Sequential([
    Input(shape=(X_train_dense.shape[1],)),
    Dense(64, activation="relu"),
    Dropout(0.2),
    Dense(32, activation="relu"),
    Dense(1, activation="sigmoid")
])

ann_model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

ann_stop = EarlyStopping(
    monitor="val_loss", patience=5, restore_best_weights=True
)

ann_history = ann_model.fit(
    X_train_dense, y_train,
    epochs=40,
    batch_size=8,
    validation_split=0.2,
    callbacks=[ann_stop],
    verbose=1
)
training_histories["ANN"] = ann_history

ann_prob = ann_model.predict(X_test_dense, verbose=0).ravel()
ann_pred = (ann_prob >= 0.5).astype(int)
evaluate_model("ANN", y_test, ann_pred)
```

8. Complete Python Implementation - Part 3

Continue the same Python file.

```
# STEP 7: Prepare text input for RNN and LSTM
MAX_WORDS = 3000
MAX_LENGTH = 40

train_text = tf.constant(X_train, dtype=tf.string)
test_text = tf.constant(X_test, dtype=tf.string)

def create_sequence_vectorizer():
    vectorizer = TextVectorization(
        max_tokens=MAX_WORDS,
        standardize="lower_and_strip_punctuation",
        split="whitespace",
        output_mode="int",
        output_sequence_length=MAX_LENGTH
    )
    vectorizer.adapt(train_text)
    return vectorizer

# Common function to build RNN or LSTM
def build_sequence_model(model_type):
    vectorizer = create_sequence_vectorizer()
    vocab_size = len(vectorizer.get_vocabulary())

    layers = [
        Input(shape=(), dtype=tf.string),
        vectorizer,
        Embedding(
            input_dim=vocab_size,
            output_dim=64,
            mask_zero=True
        )
    ]

    if model_type == "RNN":
        layers.append(SimpleRNN(32, activation="tanh"))
    elif model_type == "LSTM":
        layers.append(LSTM(32, activation="tanh"))
    else:
        raise ValueError("model_type must be RNN or LSTM")

    layers.append(Dense(1, activation="sigmoid"))
    model = tf.keras.Sequential(layers)

    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )
    return model

# STEP 8: RECURRENT NEURAL NETWORK
print("\\nTRAINING RNN")
rnn_model = build_sequence_model("RNN")
rnn_stop = EarlyStopping(
    monitor="val_loss", patience=5, restore_best_weights=True
)
rnn_history = rnn_model.fit(
    train_text, y_train,
    epochs=40,
    batch_size=8,
    validation_split=0.2,
    callbacks=[rnn_stop],
    verbose=1
)
training_histories["RNN"] = rnn_history

rnn_prob = rnn_model.predict(test_text, verbose=0).ravel()
rnn_pred = (rnn_prob >= 0.5).astype(int)
evaluate_model("RNN", y_test, rnn_pred)
```

8. Complete Python Implementation - Part 4

Continue the same Python file.

```
# STEP 9: LONG SHORT-TERM MEMORY
print("\\nTRAINING LSTM")
lstm_model = build_sequence_model("LSTM")
lstm_stop = EarlyStopping(
    monitor="val_loss", patience=5, restore_best_weights=True
)
lstm_history = lstm_model.fit(
    train_text, y_train,
    epochs=40,
    batch_size=8,
    validation_split=0.2,
    callbacks=[lstm_stop],
    verbose=1
)
training_histories["LSTM"] = lstm_history

lstm_prob = lstm_model.predict(test_text, verbose=0).ravel()
lstm_pred = (lstm_prob >= 0.5).astype(int)
evaluate_model("LSTM", y_test, lstm_pred)

# STEP 10: Final performance comparison
results_df = pd.DataFrame(results)
print("\\nFINAL MODEL COMPARISON")
print(results_df.round(4).to_string(index=False))

# STEP 11: Performance comparison graph
ax = results_df.set_index("Model")[
    ["Accuracy", "Precision", "Recall", "F1-Score"]
].plot(kind="bar", figsize=(11, 6))

ax.set_title("ML and DL Models - Performance Comparison")
ax.set_ylabel("Performance Score")
ax.set_xlabel("Models")
ax.set_ylim(0, 1.1)
plt.xticks(rotation=20)
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()

# STEP 12: Confusion matrices for all five models
fig, axes = plt.subplots(2, 3, figsize=(12, 8))

for ax, (name, cm) in zip(axes.flat, confusion_matrices.items()):
    sns.heatmap(
        cm, annot=True, fmt="d", cmap="Blues",
        xticklabels=["Negative", "Positive"],
        yticklabels=["Negative", "Positive"],
        ax=ax
    )
    ax.set_title(name)
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")

axes.flat[-1].axis("off")
plt.tight_layout()
plt.show()
```

8. Complete Python Implementation - Part 5

Add the final plotting and results-saving section.

```
# STEP 13: Training loss comparison for ANN, RNN and LSTM
plt.figure(figsize=(9, 5))

for name, history in training_histories.items():
    plt.plot(history.history["loss"], label=name)

plt.title("Deep Learning Models - Training Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# STEP 14: Save comparison results
results_df.to_csv("model_comparison_results.csv", index=False)
print("\\nComparison results saved successfully!")
```

Keep all five code parts in the same Python file, in the order shown.

9. How to Execute the Program

1. Create a folder named NLP_Model_Comparison.
2. Place sentiment_dataset.csv and compare_ml_dl_nlp.py inside the folder.
3. Open Command Prompt, PowerShell or the VS Code terminal.
4. Navigate to the folder using the cd command.
5. Install the required libraries using pip.
6. Run the Python file and allow all five models to train and evaluate.

Terminal commands

```
cd path\to\NLP_Model_Comparison
python -m pip install --upgrade pip
pip install pandas numpy scikit-learn tensorflow matplotlib seaborn
python compare_ml_dl_nlp.py
```

10. Understanding the Evaluation Metrics

Metric	Meaning
Accuracy	Fraction of all test reviews classified correctly.
Precision	Of reviews predicted as positive, the fraction that are actually positive.
Recall	Of actual positive reviews, the fraction correctly identified.
F1-score	Harmonic mean of precision and recall; balances the two measures.

11. Understanding the Generated Visualizations

Visualization	What to observe
Performance comparison bar chart	Compares accuracy, precision, recall and F1-score across the five models.
Confusion matrices	Show correct and incorrect predictions for Negative and Positive reviews for each model.
Training loss graph	Compares training loss across ANN, RNN and LSTM. Lower training loss alone does not prove better tes

12. Output Files

File / Output	Description
model_comparison_results.csv	Stores the final Accuracy, Precision, Recall and F1-score for each model.
Performance comparison graph	Displayed in a Matplotlib window.
Five confusion matrices	Displayed together in a 2-by-3 grid; the last panel is left empty.
Training loss plot	Displayed for ANN, RNN and LSTM.

Important: Actual scores are generated when the program runs. With only 100 synthetic reviews, the results can vary substantially and should be treated as a demonstration, not as a reliable ranking of models.

13. Suggested Laboratory Experiment

Title: Comparative Analysis of Machine Learning and Deep Learning Models for NLP.

Aim: To implement and compare Naive Bayes, SVM, ANN, RNN and LSTM models for sentiment classification, and visualize their performance using accuracy, precision, recall and F1-score.

Learning Outcomes

1. Understand machine learning and deep learning approaches for NLP.
2. Implement five classification models for sentiment analysis.
3. Understand the differences between TF-IDF and embedding-based representations.
4. Evaluate models using standard classification metrics.
5. Visualize and interpret comparative model performance.

14. Experimental Considerations

The program uses the same stratified train-test split for every model. TF-IDF and neural text vocabularies are fitted using training text only. However, this remains a comparison of complete pipelines: Naive Bayes, SVM and ANN use TF-IDF, whereas RNN and LSTM use sequence embeddings. The differences in feature representation are part of the comparison.

For a stronger study, use a larger, diverse and independently labelled dataset. Consider repeated stratified cross-validation, training time, model complexity and held-out test results. Do not select a model solely because it has the lowest training loss or highest training accuracy.

15. Teaching Sequence

Order	Classroom activity
1	Explain the dataset, target labels and common train-test split.
2	Demonstrate TF-IDF and train Naive Bayes and SVM.
3	Explain ANN layers and train the feedforward neural network.
4	Explain sequence embeddings, hidden states and train the RNN.
5	Explain LSTM gates and train the LSTM model.
6	Compare metrics, confusion matrices and training curves; discuss limitations.

16. Documentation

Scikit-learn: scikit-learn.org
TensorFlow: tensorflow.org
Keras: keras.io